

CareSync Complete Understanding Guide – Part 2

Terraform Concepts and AWS Services in Detail

Terraform: The Foundation of the Entire Project

Before discussing AWS services, it is important to understand Terraform because Terraform is the technology that created, configured, connected, and manages every AWS resource in the CareSync infrastructure.

Many beginners think Terraform is simply a tool that creates AWS resources. That understanding is incomplete.

Terraform is actually an Infrastructure as Code platform.

Infrastructure means servers, databases, networking components, load balancers, storage systems, security controls, monitoring systems, and every cloud resource required to run an application.

Traditionally, infrastructure engineers manually created resources through the AWS Console. They clicked buttons, selected options, created networks, attached security groups, and configured databases one resource at a time.

This process worked for small environments but became extremely difficult in enterprise environments.

Imagine a company running hundreds of AWS resources.

If a new engineer joins the company and needs to recreate the infrastructure, manually clicking through the console becomes almost impossible.

There is also a high chance of mistakes.

One missing security group rule or one incorrectly configured subnet can break the application.

Terraform solves this problem by allowing infrastructure to be described as code.

Instead of saying:

"I clicked these buttons in AWS"

we can say:

"I wrote this Terraform code."

The code becomes the documentation.

The code becomes the deployment process.

The code becomes the source of truth.

This is why Infrastructure as Code is considered one of the most important practices in modern cloud engineering.

In CareSync, Terraform is responsible for creating:

- VPC
- Subnets
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups
- Application Load Balancers
- Auto Scaling Groups
- Launch Templates
- EC2 Instances
- RDS PostgreSQL
- S3 Bucket
- KMS Key
- Secrets Manager Secret
- IAM Roles
- CloudWatch Monitoring
- SNS Notifications
- Lambda Functions
- EventBridge Scheduler
- CloudFront
- WAF

Without Terraform, every one of these resources would have to be created manually.

Terraform Modules

One of the most important Terraform concepts used in CareSync is Modules.

A module can be thought of as a reusable package of infrastructure.

Imagine writing the entire infrastructure inside a single file.

The file would quickly grow into thousands of lines.

Finding a problem would become extremely difficult.

For example, if a database issue occurs, an engineer would have to search through a massive file to locate the RDS configuration.

Instead, Terraform allows us to organize infrastructure into modules.

Each module has a specific responsibility.

The VPC module manages networking.

The Security Group module manages firewall rules.

The ALB module manages load balancing.

The RDS module manages the database.

The Lambda module manages serverless functions.

This approach follows the software engineering principle known as Separation of Concerns.

Each module focuses on one responsibility.

This improves maintainability, readability, scalability, and troubleshooting.

This is why CareSync contains separate modules for networking, security, storage, monitoring, automation, and content delivery.

In enterprise environments, modules are often reused across multiple projects.

A company may create a VPC module once and reuse it in dozens of different applications.

Terraform State

Terraform state is one of the most important concepts to understand.

Terraform itself does not continuously monitor AWS resources.

Instead, Terraform keeps a record of resources inside a state file.

The state file acts like Terraform's memory.

It contains information about every resource Terraform manages.

When Terraform executes a plan, it compares three things:

The code.

The state file.

The actual AWS environment.

By comparing these three sources, Terraform determines what changes are required.

Without the state file Terraform loses its memory.

It would no longer know which resources belong to the project.

This is why protecting the state file is critical.

Remote Backend with S3

Initially Terraform stores state locally.

This works for learning projects but becomes dangerous in real environments.

Suppose your laptop crashes.

The local state file disappears.

The AWS infrastructure still exists.

However Terraform no longer knows what resources it created.

This can lead to significant operational problems.

To solve this problem CareSync uses a Remote Backend.

The state file is stored in Amazon S3.

S3 acts as centralized storage.

Now both your personal laptop and company laptop access the same state.

The state becomes independent of any individual machine.

This is exactly how professional teams operate.

The S3 backend is effectively the single source of truth for infrastructure management.

DynamoDB State Locking

Imagine two engineers working simultaneously.

Engineer A executes:

terraform apply

Engineer B executes:

terraform apply

at exactly the same time.

Both attempt to modify the state file.

The result could be corruption.

Terraform solves this using DynamoDB locking.

Before Terraform modifies infrastructure, it creates a lock entry inside DynamoDB.

While the lock exists, no other Terraform operation can modify the state.

Once the operation completes, the lock is removed.

This mechanism prevents concurrent changes and protects infrastructure integrity.

This is why S3 and DynamoDB are commonly deployed together.

S3 stores the state.

DynamoDB protects the state.

Amazon VPC

The Virtual Private Cloud is the foundation of the entire AWS architecture.

A VPC can be compared to a private data center inside AWS.

Every server, database, load balancer, and application component lives inside the VPC.

Without a VPC, resources would have no secure network environment.

In CareSync we created a custom VPC instead of using the default VPC.

This provides complete control over networking and security.

From a business perspective, healthcare applications handle sensitive patient data.

The company must control exactly which systems can communicate with each other.

A custom VPC allows that control.

The VPC is divided into multiple subnets.

Each subnet serves a specific purpose.

This network segmentation reduces risk and improves security.

Public and Private Subnets

A subnet is a smaller network inside the VPC.

CareSync uses multiple subnet types.

Public subnets host internet-facing resources.

Private subnets host application servers.

Database subnets host PostgreSQL.

This separation is extremely important.

In the real world, exposing databases directly to the internet is considered a major security risk.

By placing the database in isolated private subnets, we reduce the attack surface significantly.

This architecture follows enterprise security best practices.

Security Groups

Security Groups act as virtual firewalls.

Every resource receives a Security Group.

The Security Group determines what traffic is allowed and what traffic is denied.

For example:

The frontend server accepts requests from the load balancer.

The backend server accepts API requests.

The database only accepts PostgreSQL traffic from backend components.

This creates multiple security layers.

A hacker who compromises one system still cannot freely access every other system.

In healthcare environments where patient information is involved, such layered security is extremely important.

Application Load Balancer

A load balancer distributes traffic across multiple servers.

Imagine a hospital portal receiving thousands of requests.

If only one server exists, that server becomes overloaded.

Performance suffers.

Availability decreases.

The Application Load Balancer solves this problem.

Instead of users connecting directly to EC2 instances, they connect to the load balancer.

The load balancer intelligently distributes traffic.

In CareSync, path-based routing is used.

Requests for the frontend go to frontend servers.

Requests beginning with /api are routed to backend servers.

This allows a single entry point while supporting multiple application tiers.

The load balancer improves scalability, reliability, and fault tolerance.